Family Music Scrobbler PWA — Developer README

Family Music Scrobbler PWA - Developer README

A modern, offline-capable Progressive Web App for family music analytics, powered by FastAPI, PostgreSQL, React, and APScheduler. Designed for long-term ingestion of Spotify listening history, multi-device PWA usage, and home-lab deployment on an Intel NUC with Synology NAS backups.

📁 Project Structure

```
backend/
  app/
    main.py
    api/
    auth/
    db/
      session.py
      models/
      migrations/
    workers/
    schemas/
    utils/
frontend/
  src/
    components/
    pages/
    hooks/
    lib/
    service-worker.js
    manifest.json
docker/
  dev/
  prod/
scripts/
  verify_backup.sh
infra/
  docker-compose.yml
  docker-compose.override.yml
```

🧱 Core Technologies

- **Backend:** FastAPI (Python 3.11+)
- **Frontend:** React + PWA
- **Database:** PostgreSQL (partitioned monthly)
- **Ingestion Worker:** APScheduler
- **Auth:** Local auth + Spotify OAuth
- **Offline Sync:** IndexedDB + Background Sync API
- **Deployment:** Docker + Docker Compose
- **Secrets:** AES-encrypted refresh tokens via Docker secrets
- **Backups:** pg_dump + rsync to Synology DS223

🚀 Development Environment (Fedora 43)

All development happens locally on:

- Lenovo ThinkPad P14s Gen 1
- Fedora 43
- VS Code
- Local Docker Compose
- GitHub repo: Notascooby25

Local Setup

1. Clone the repo
2. Copy .env.example → .env
3. Start dev stack:

```
docker compose -f infra/docker-compose.override.yml up --build
```

1. Start frontend:

```
npm install
npm run dev
```

1. Start backend:

```
uvicorn backend.app.main:app --reload
```

🗄️ Database Schema Overview

Scrobbles Table

- played_at TIMESTAMPTZ NOT NULL

- track_id UUID REFERENCES tracks(id)
- UNIQUE(user_id, spotify_play_id)
- Partitioned by month: scrobbles_YYYY_MM

Indexes

- (user_id, played_at) - compound B-Tree
- (spotify_play_id) - dedupe enforcement

🔄 Ingestion Worker (APScheduler)

The backend includes a background worker responsible for polling Spotify:

- Poll interval: **5-10 minutes**
- Endpoint: user-read-recently-played
- Deduplication via spotify_play_id
- Refresh token lifecycle:
  - AES encryption at rest
  - Automatic rotation
  - Exponential backoff on 429/5xx

Worker state is stored in the database via last_polled.

📱 PWA Architecture

Caching Strategy

- **Cache-first:** App shell, CSS, JS
- **Network-first:** API calls

Offline Sync

- IndexedDB queue
- Background Sync API
- Automatic replay when connection restores

Push Notifications

- VAPID keys stored securely
- Used for leaderboard updates and ingestion alerts

🧩 Frontend Features

- Recent tracks dashboard
- Weekly leaderboard
- Family profiles
- Consent-based tracking
- Data export (JSON/CSV)
- Account deletion with cascading revocation

📦 Deployment Environment (Intel NUC)

Production runs on:

- Intel NUC (Core i5-4250U, 8GB RAM)
- Fedora Server
- Docker Compose (production)
- Synology DS223 NAS for backups

Production Deployment

```
docker compose -f infra/docker-compose.yml up --build -d
```

Secrets

AES keys stored via:

```
/run/secrets/refresh_token_key
```

🛡️ Backup & Verification

The script scripts/verify_backup.sh performs:

1. pg_dump
2. rsync to Synology DS223
3. Spins up a temporary container
4. Restores the dump
5. Runs integrity checks

This ensures backups are valid and restorable.

📊 Observability

Backend exposes:

- /healthz - health check
- Structured JSON logs
- Worker heartbeat logs
- Ingestion metrics

🔧 Development Workflow (Planner Agent)

All changes must follow the VS Code planner agent protocol:

1. Impact Analysis
2. Regression Check
3. Step-by-step Plan
4. User Approval
5. Code Generation
6. Verification Commands Suggested

This ensures architectural consistency and prevents accidental regressions.

🔐 Privacy & Data Governance

- Explicit consent required during onboarding
- Users may opt out of family leaderboard
- Full account deletion cascades:
  - scrobbles
  - tokens
  - profile
- Export available in JSON/CSV

🧭 Roadmap (Phased Execution)

1. Environment Setup
2. Database + Backend Foundation
3. Core Logic + Ingestion
4. Frontend + PWA
5. Production Deployment + Backups